

Lista de Exercícios de Fixação – Questões Teóricas

1. Explique a diferença entre `mysql2` e `mysql2/promise`.

O pacote `mysql2` é utilizado para conectar aplicações Node.js ao banco MySQL, oferecendo suporte a callbacks para lidar com operações assíncronas.

Já o `mysql2/promise` é uma versão baseada em Promises, permitindo o uso de `async/await`, o que torna o código mais moderno, legível e fácil de manter.

2. Quais as vantagens de usar variáveis de ambiente na conexão com o banco de dados?

- Proteção de dados sensíveis (usuário, senha, host)
 - Facilidade de configuração entre ambientes (desenvolvimento, teste e produção)
 - Evita exposição de credenciais no código-fonte
 - Melhora a organização e manutenção do projeto
-

3. O que é um `prepared statement` e como ele ajuda na segurança da aplicação?

Prepared Statement é uma forma de executar consultas SQL utilizando parâmetros ao invés de concatenar valores diretamente na query.

Ele ajuda na segurança porque evita ataques de SQL Injection, garantindo que os dados do usuário sejam tratados como valores e não como comandos SQL.

4. Por que é recomendável usar conexões em pool em aplicações Node.js?

O pool de conexões permite reutilizar conexões com o banco de dados ao invés de criar uma nova a cada requisição.

Vantagens:

- Melhor performance
 - Redução de sobrecarga no banco
 - Maior escalabilidade
 - Gerenciamento eficiente de múltiplas conexões
-

5. Descreva o papel do módulo dotenv em projetos Node.js.

O `dotenv` é um pacote que permite carregar variáveis de ambiente a partir de um arquivo `.env`.

Ele é utilizado para armazenar configurações sensíveis, como credenciais de banco de dados, mantendo essas informações fora do código principal.

6. Qual é a estrutura básica para realizar uma consulta SELECT com o pacote mysql2?

A estrutura básica envolve:

1. Criar uma conexão com o banco
2. Executar a query SELECT
3. Receber e tratar os resultados

Exemplo conceitual:

- Conectar ao banco
 - Executar `SELECT * FROM tabela`
 - Retornar os dados obtidos
-

7. Liste três boas práticas ao conectar o Node.js com o MySQL.

- Utilizar variáveis de ambiente para credenciais
 - Usar prepared statements para evitar SQL Injection
 - Implementar pool de conexões
 - Tratar erros corretamente
 - Organizar o código em módulos
-

8. Explique o conceito de transações no MySQL e como ele pode ser implementado em Node.js.

Transações são um conjunto de operações SQL executadas como uma única unidade. Elas garantem que todas as operações sejam concluídas com sucesso ou nenhuma seja aplicada.

No Node.js, podem ser implementadas utilizando comandos como:

- `BEGIN` (iniciar transação)
 - `COMMIT` (confirmar alterações)
 - `ROLLBACK` (desfazer alterações em caso de erro)
-

9. Quais erros podem ocorrer ao conectar ao MySQL, e como tratá-los no Node.js?

Erros comuns:

- Falha de conexão
- Credenciais incorretas
- Banco indisponível
- Timeout

Tratamento:

- Uso de `try/catch` com `async/await`
 - Tratamento de erros em Promises
 - Logs para monitoramento
 - Mensagens de erro claras
-

10. Por que é importante organizar o código em módulos ao trabalhar com bancos de dados?

A organização em módulos permite:

- Melhor manutenção do código
- Reutilização de funções
- Separação de responsabilidades
- Código mais limpo e escalável

Exemplo: separar conexão (`db.js`) das operações (`usuarios.js`).
